

Pandas Basics to Advanced

You said:

pandas all concept explain with code and basics to Advanced

ChatGPT said:

I'll organize Pandas from beginner to advanced in easy words, with one reusable dataset, practical code, common mistakes, and interview points. I'm checking the current official Pandas documentation so the method names and recommendations are up to date.

ChatGPT said:

Pandas: Basics to Advanced with Easy Code

Pandas is a Python library used to clean, organize, analyze, and transform table-like data. Think of it as **Excel + SQL inside Python**.

The current official documentation is Pandas 3.0.4. Its two main structures are **Series** and **DataFrame**.
[Official Pandas beginner guide](#)

1. Installation and Import

```
pip install pandas
```

For Excel files:

```
pip install pandas openpyxl
```

Import Pandas:

```
import pandas as pd  
  
print(pd.__version__)
```

Part 1: Pandas Data Structures

2. Series

A **Series** is a single column of data.

```
import pandas as pd

salary = pd.Series([40000, 50000, 60000, 70000])

print(salary)
```

Output:

```
0    40000
1    50000
2    60000
3    70000
```

Custom index:

```
salary = pd.Series(
    [40000, 50000, 60000],
    index=["Aamir", "Ravi", "Neha"]
)

print(salary["Aamir"])
```

Output:

```
40000
```

3. DataFrame

A **DataFrame** is a complete table containing rows and columns.

```
data = {
    "id": [101, 102, 103, 104, 105, 106],
    "name": ["Aamir", "Sara", "Ravi", "Neha", "Kabir", "Sara"],
    "department": ["AI", "HR", "AI", "Sales", "Sales", "HR"],
    "city": ["Delhi", "Jaipur", "Delhi", "Mumbai", "Jaipur", "Jaipur"],
    "salary": [60000, 45000, 70000, None, 52000, 45000],
    "experience": [2, 3, 5, 1, 4, 3],
    "rating": [4, 5, 4, 3, 5, 5],
    "joining_date": [
        "2024-01-10",
        "2023-05-15",
        "2021-07-20",
        "2025-02-01",
        "2022-09-12",
        "2023-05-15"
    ]
}

df = pd.DataFrame(data)

print(df)
```

4. Series vs DataFrame

Feature	Series	DataFrame
Dimension	One-dimensional	Two-dimensional
Looks like	One column	Complete table
Example	Salary column	Employee table
Selection	df["salary"]	df[["name", "salary"]]

Part 2: Understanding the Data

5. Basic Inspection Methods

```
print(df.head())    # First 5 rows
print(df.head(2))   # First 2 rows
print(df.tail())    # Last 5 rows

print(df.shape)     # Number of rows and columns
print(df.columns)    # Column names
print(df.index)      # Row indexes
print(df.dtypes)     # Data type of every column
print(df.info())     # Complete structure
```

Statistical summary

```
print(df.describe())
```

It shows:

- Count
- Mean
- Standard deviation
- Minimum
- Maximum
- Quartiles

Include text columns:

```
print(df.describe(include="all"))
```

6. Important DataFrame Properties

```
print(df.size)      # Total cells
print(df.ndim)       # Number of dimensions
print(df.empty)      # Is DataFrame empty?
print(df.values)     # Data as NumPy array
```


Part 3: Selecting Data

7. Selecting Columns

One column returns a **Series**:

```
names = df["name"]  
print(names)
```

Multiple columns return a **DataFrame**:

```
result = df[["name", "salary", "department"]]  
print(result)
```

8. **loc** – Select Using Labels

Syntax:

```
df.loc[row_labels, column_labels]
```

Examples:

```
print(df.loc[0])  
  
print(df.loc[0, "name"])  
  
print(df.loc[0:2, ["name", "salary"]])
```

Important: In **loc[0:2]**, row **2** is included.

9. **iloc** – Select Using Positions

Syntax:

```
df.iloc[row_positions, column_positions]
```

```
print(df.iloc[0])      # First row  
print(df.iloc[0, 1])   # First row, second column  
print(df.iloc[0:3, 1:4]) # Rows 0–2 and columns 1–3
```

Important: In **iloc[0:3]**, position **3** is not included.

10. **at** and **iat**

Use them when you need one single value.

```
print(df.at[0, "name"]) # Label-based  
print(df.iat[0, 1])    # Position-based
```

11. Conditional Filtering

Employees with salary above ₹50,000:

```
result = df[df["salary"] > 50000]  
print(result)
```

Multiple conditions:

```
result = df[  
    (df["salary"] > 50000) &  
    (df["department"] == "AI")  
]  
  
print(result)
```

Use:

- **&** for AND
- **|** for OR
- **~** for NOT

Always put each condition inside parentheses.

```
result = df[  
    (df["city"] == "Delhi") |  
    (df["city"] == "Jaipur")  
]
```

12. `isin()`, `between()` and `query()`

```
# City is Delhi or Jaipur
print(df[df["city"].isin(["Delhi", "Jaipur"])])

# Salary between 45,000 and 60,000
print(df[df["salary"].between(45000, 60000)])

# SQL-like filtering
print(df.query("salary > 50000 and department == 'AI'"))
```

Using a Python variable in `query()`:

```
minimum_salary = 50000

result = df.query("salary >= @minimum_salary")
print(result)
```

Part 4: Adding, Updating and Deleting Data

13. Add a New Column

```
df["annual_salary"] = df["salary"] * 12
```

Conditional column:

```
df["high_rating"] = df["rating"] >= 4
```

Using `assign()`:

```
new_df = df.assign(
    bonus=df["salary"] * 0.10,
    monthly_salary=df["salary"] / 12
)
```

14. Update Values

Update one value:

```
df.loc[0, "salary"] = 65000
```

Update multiple matching rows:

```
df.loc[df["department"] == "AI", "rating"] = 5
```

15. Rename Columns

```
df = df.rename(columns={  
    "name": "employee_name",  
    "city": "location"  
})
```

Rename all columns:

```
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

16. Delete Rows and Columns

Delete a column:

```
df = df.drop(columns=["high_rating"])
```

Delete multiple columns:

```
df = df.drop(columns=["annual_salary", "monthly_salary"])
```

Delete a row:

```
df = df.drop(index=0)
```

Avoid unnecessary `inplace=True`; assigning the result back is usually clearer.

Part 5: Sorting and Ranking

17. Sort Data

Ascending order:

```
df = df.sort_values("salary")
```

Descending order:

```
df = df.sort_values("salary", ascending=False)
```

Multiple columns:

```
df = df.sort_values(  
    ["department", "salary"],  
    ascending=[True, False]  
)
```

Sort by index:

```
df = df.sort_index()
```

18. Highest and Lowest Values

```
print(df.nlargest(3, "salary"))  
print(df.nsmallest(2, "salary"))
```

19. Ranking

```
df["salary_rank"] = df["salary"].rank(  
    method="dense",  
    ascending=False  
)
```

dense ranking produces:

```
1, 2, 2, 3
```

Part 6: Missing Values

Pandas represents missing information using values such as **NaN**, **NaT**, or **pd.NA**. [Official missing-data guide](#)

20. Find Missing Values

```
print(df.isna())  
print(df.isna().sum())
```

Find non-missing values:

```
print(df.notna().sum())
```

Rows where salary is missing:

```
print(df[df["salary"].isna()])
```

21. Fill Missing Values

Fill with a fixed value:

```
df["salary"] = df["salary"].fillna(0)
```

Fill with mean:

```
df["salary"] = df["salary"].fillna(  
    df["salary"].mean()  
)
```

Fill with median:

```
df["salary"] = df["salary"].fillna(  
    df["salary"].median()  
)
```

Median is usually safer when data contains extreme values.

Fill using department median:

```
department_median = (  
    df.groupby("department")["salary"]  
    .transform("median")  
)  
  
df["salary"] = df["salary"].fillna(department_median)
```

Forward and backward filling:

```
df["salary"] = df["salary"].ffill()  
df["salary"] = df["salary"].bfill()
```

Interpolation:

```
df["salary"] = df["salary"].interpolate()
```

22. Remove Missing Values

Remove rows containing any missing value:

```
clean_df = df.dropna()
```

Remove rows only when salary is missing:

```
clean_df = df.dropna(subset=["salary"])
```

Remove columns containing missing values:

```
clean_df = df.dropna(axis="columns")
```

Part 7: Duplicate Data

23. Find Duplicates

```
print(df.duplicated())  
print(df.duplicated().sum())
```

Check duplicates using selected columns:

```
print(  
    df.duplicated(  
        subset=["name", "department"],  
        keep=False  
    )  
)
```

24. Remove Duplicates

```
df = df.drop_duplicates()
```

Keep the last occurrence:

```
df = df.drop_duplicates(  
    subset=["name", "department"],  
    keep="last"  
)
```

Part 8: Data Types and Cleaning

25. Change Data Type

```
df["experience"] = df["experience"].astype("int64")  
df["salary"] = df["salary"].astype("Float64")  
df["name"] = df["name"].astype("string")
```

Convert safely to numeric:

```
df["salary"] = pd.to_numeric(  
    df["salary"],  
    errors="coerce"  
)
```

Invalid values become missing values when `errors="coerce"` is used.

26. Clean Text Data

```
df["name"] = df["name"].str.strip()
df["name"] = df["name"].str.title()
df["city"] = df["city"].str.lower()
```

Other useful string operations:

```
print(df["name"].str.contains("amir", case=False))
print(df["name"].str.startswith("A"))
print(df["name"].str.len())
print(df["name"].str.replace(" ", "_"))
print(df["name"].str.split())
```

27. Replace Values

```
df["department"] = df["department"].replace({
    "AI": "Artificial Intelligence",
    "HR": "Human Resources"
})
```

Replace several invalid values with missing data:

```
df = df.replace(["NA", "N/A", "unknown", ""], pd.NA)
```

28. Detect Outliers Using IQR

```
q1 = df["salary"].quantile(0.25)
q3 = df["salary"].quantile(0.75)

iqr = q3 - q1

lower_limit = q1 - 1.5 * iqr
upper_limit = q3 + 1.5 * iqr

outliers = df[
    (df["salary"] < lower_limit) |
    (df["salary"] > upper_limit)
]

print(outliers)
```

Remove outliers:

```
df_without_outliers = df[
    df["salary"].between(lower_limit, upper_limit)
]
```

Part 9: Statistical Operations

29. Common Statistical Methods

```
print(df["salary"].count())
print(df["salary"].sum())
print(df["salary"].mean())
print(df["salary"].median())
print(df["salary"].mode())
print(df["salary"].min())
print(df["salary"].max())
print(df["salary"].std())
print(df["salary"].var())
print(df["salary"].quantile(0.75))
```

30. Multiple Aggregations

```
result = df["salary"].agg([
    "count",
    "sum",
    "mean",
    "median",
    "min",
    "max"
])

print(result)
```

31. Correlation

```
numeric_df = df.select_dtypes(include="number")

print(numeric_df.corr())
```

Correlation range:

- **+1**: Strong positive relationship
- **0**: No linear relationship
- **-1**: Strong negative relationship

Correlation does not prove that one variable causes another.

Part 10: GroupBy

`groupby()` follows the idea: split data into groups, apply a calculation, and combine the results. [Official GroupBy guide](#)

32. Basic GroupBy

Average salary by department:

```
result = df.groupby("department")["salary"].mean()
print(result)
```

Total salary by city:

```
result = df.groupby("city")["salary"].sum()
print(result)
```

33. GroupBy with Multiple Columns

```
result = (
    df.groupby(["department", "city"])["salary"]
      .mean()
)

print(result)
```

34. Named Aggregation

```
summary = (  
    df.groupby("department", as_index=False)  
        .agg(  
            employee_count=("id", "count"),  
            average_salary=("salary", "mean"),  
            total_salary=("salary", "sum"),  
            maximum_salary=("salary", "max")  
        )  
)  
  
print(summary)
```

35. **agg()** vs **transform()** vs **apply()**

agg()

Returns a summary for each group.

```
df.groupby("department")["salary"].agg("mean")
```

transform()

Returns the same number of rows as the original DataFrame.

```
df["department_average"] = (  
    df.groupby("department")["salary"]  
        .transform("mean")  
)
```

apply()

Used for custom logic. It is flexible but often slower.

```
def top_employee(group):  
    return group.nlargest(1, "salary")  
  
result = (  
    df.groupby("department", group_keys=False)  
        .apply(top_employee)  
)
```

Prefer built-in operations such as **sum**, **mean**, **agg**, and **transform** whenever possible.

36. Filter Complete Groups

Keep departments whose average salary is above ₹50,000:

```
result = df.groupby("department").filter(  
    lambda group: group["salary"].mean() > 50000  
)
```

Part 11: Combining DataFrames

37. `concat()`

Used to place DataFrames above or beside each other.

Add rows:

```
combined = pd.concat(  
    [df1, df2],  
    ignore_index=True  
)
```

Add columns:

```
combined = pd.concat(  
    [df1, df2],  
    axis="columns"  
)
```

38. `merge()`

`merge()` works like SQL joins.

```
department_data = pd.DataFrame({
    "department": ["AI", "HR", "Sales"],
    "manager": ["Rahul", "Priya", "Amit"]
})

result = df.merge(
    department_data,
    on="department",
    how="left",
    validate="many_to_one"
)

print(result)
```

Join types

Join Meaning

inner Only matching rows

left All left rows and matching right rows

right All right rows and matching left rows

outer All rows from both tables

cross Every possible combination

Example:

```
inner_result = df.merge(department_data, how="inner")
left_result = df.merge(department_data, how="left")
outer_result = df.merge(department_data, how="outer")
```

39. **join()**

join() normally combines DataFrames using indexes.

```

left = df.set_index("department")
right = department_data.set_index("department")

result = left.join(
    right,
    how="left",
    rsuffix="_right"
)

```

Part 12: Reshaping Data

Pandas supports **pivot**, **pivot_table**, **melt**, **stack**, and **unstack** for changing data layout. [Official reshaping guide](#)

40. Pivot Table

```

table = pd.pivot_table(
    df,
    index="department",
    columns="city",
    values="salary",
    aggfunc="mean",
    fill_value=0
)

print(table)

```

41. **pivot()** vs **pivot_table()**

pivot()	pivot_table()
Does not aggregate	Can aggregate
Duplicate combinations cause an error	Handles duplicates
Mainly reshapes	Reshapes and summarizes

42. Crosstab

Count employees by department and city:

```
result = pd.crosstab(  
    df["department"],  
    df["city"],  
    margins=True  
)  
  
print(result)
```

43. Melt: Wide to Long

```
long_data = df.melt(  
    id_vars=["id", "name"],  
    value_vars=["salary", "rating"],  
    var_name="metric",  
    value_name="value"  
)  
  
print(long_data)
```

44. Stack and Unstack

```
grouped = df.groupby(  
    ["department", "city"]  
)  
.size()  
  
wide = grouped.unstack(fill_value=0)  
long_again = wide.stack()  
  
print(wide)
```

Part 13: Date and Time

Pandas provides date parsing, date arithmetic, time-based grouping, resampling, and shifting. [Official time-series guide](#)

45. Convert to Date

```
df["joining_date"] = pd.to_datetime(  
    df["joining_date"],  
    errors="coerce"  
)
```


46. Extract Date Parts

```
df["joining_year"] = df["joining_date"].dt.year
df["joining_month"] = df["joining_date"].dt.month
df["month_name"] = df["joining_date"].dt.month_name()
df["day_name"] = df["joining_date"].dt.day_name()
df["quarter"] = df["joining_date"].dt.quarter
```

47. Filter Dates

```
result = df[
    df["joining_date"] >= pd.Timestamp("2023-01-01")
]
```

Filter a date range:

```
result = df[
    df["joining_date"].between(
        "2023-01-01",
        "2024-12-31"
    )
]
```

48. Date Difference

```
today = pd.Timestamp.today().normalize()

df["days_in_company"] = (
    today - df["joining_date"]
).dt.days
```

49. Resampling

```
sales = pd.DataFrame({
    "date": pd.to_datetime([
        "2026-01-05",
        "2026-01-20",
        "2026-02-10",
        "2026-03-15"
    ]),
    "amount": [1000, 1500, 2000, 2500]
})

monthly_sales = (
    sales.resample("MS", on="date")["amount"]
    .sum()
)

print(monthly_sales)
```

Part 14: Window Functions

50. Rolling Calculation

Three-row moving average:

```
df = df.sort_values("joining_date")

df["rolling_salary"] = (
    df["salary"]
    .rolling(window=3, min_periods=1)
    .mean()
)
```

51. Expanding Calculation

Cumulative average from the first row:

```
df["expanding_average"] = (
    df["salary"]
    .expanding()
    .mean()
)
```

52. Cumulative Functions

```
df["running_total"] = df["salary"].cumsum()
df["running_max"] = df["salary"].cummax()
df["running_min"] = df["salary"].cummin()
df["running_product"] = df["experience"].cumprod()
```

53. Previous and Next Values

```
df["previous_salary"] = df["salary"].shift(1)
df["next_salary"] = df["salary"].shift(-1)
```

Percentage change:

```
df["salary_change_percent"] = (
    df["salary"].pct_change() * 100
)
```

54. Exponential Moving Average

```
df["ema_salary"] = (
    df["salary"]
    .ewm(span=3, adjust=False)
    .mean()
)
```

Part 15: Functions in Pandas

55. `map()`

Used mainly for element-wise transformation of a Series.

```
department_codes = {  
    "AI": 1,  
    "HR": 2,  
    "Sales": 3  
}  
  
df["department_code"] = (  
    df["department"].map(department_codes)  
)
```

56. `apply()`

Apply a function to rows or columns.

```
def create_profile(row):  
    return f'{row["name"]} works in {row["department"]}'  
  
df["profile"] = df.apply(create_profile, axis=1)
```

- `axis=0`: Work column-wise
- `axis=1`: Work row-wise

57. Lambda Function

```
df["salary_status"] = df["salary"].apply(  
    lambda value: "High" if value >= 60000 else "Normal"  
)
```

Vectorized code is normally faster:

```
df["annual_salary"] = df["salary"] * 12
```

58. `pipe()` and Method Chaining

```
def clean_columns(data):  
    data = data.copy()  
    data.columns = data.columns.str.lower().str.strip()  
    return data  
  
result = (  
    df.pipe(clean_columns)  
    .drop_duplicates()  
    .sort_values("salary", ascending=False)  
    .reset_index(drop=True)  
)
```

Method chaining makes a data-processing pipeline easier to read.

Part 16: Advanced Columns

59. `cut()` – Create Fixed Ranges

```
df["salary_level"] = pd.cut(  
    df["salary"],  
    bins=[0, 45000, 60000, 100000],  
    labels=["Low", "Medium", "High"]  
)
```

60. `qcut()` – Create Equal-Sized Groups

```
df["salary_quartile"] = pd.qcut(  
    df["salary"],  
    q=4,  
    labels=["Q1", "Q2", "Q3", "Q4"],  
    duplicates="drop"  
)
```

Difference:

- `cut()` uses your chosen value ranges.
- `qcut()` tries to place equal numbers of records in each group.

61. `explode()`

Convert list values into separate rows:

```
skills = pd.DataFrame({
    "name": ["Aamir", "Ravi"],
    "skills": [
        ["Python", "Pandas", "SQL"],
        ["Java", "React"]
    ]
})

result = skills.explode("skills")
print(result)
```

62. Normalize Nested JSON

```
json_data = [
    {
        "name": "Aamir",
        "contact": {
            "city": "Delhi",
            "phone": "9999999999"
        }
    }
]

flat_data = pd.json_normalize(
    json_data,
    sep="_"
)

print(flat_data)
```

Part 17: Indexing Concepts

63. Set and Reset Index

```
indexed_df = df.set_index("id")

print(indexed_df.loc[101])
```

Reset it:

```
normal_df = indexed_df.reset_index()
```

64. Reindexing

```
new_order = [105, 101, 104, 103, 102, 106]

result = indexed_df.reindex(new_order)
```

65. Automatic Index Alignment

Pandas performs mathematical operations using index labels.

```
s1 = pd.Series([10, 20], index=["a", "b"])
s2 = pd.Series([1, 2], index=["b", "c"])

print(s1 + s2)
```

Pandas matches:

- **a** with **a**
- **b** with **b**
- **c** with **c**

Missing matches become **NaN**.

Use **fill_value**:

```
result = s1.add(s2, fill_value=0)
print(result)
```

66. MultiIndex

MultiIndex means using multiple columns as an index.

```
multi_df = (
    df.set_index(["department", "city"])
    .sort_index()
)

print(multi_df.loc[("AI", "Delhi")])
```

Part 18: Categorical Data

Categorical dtype is useful when a column contains a small fixed set of repeated values, such as department, gender, rating, or education level. [Official categorical-data guide](#)

```
df["department"] = df["department"].astype("category")
```

Ordered category:

```
df["performance"] = pd.Categorical(
    ["Good", "Excellent", "Good", "Average", "Excellent", "Good"],
    categories=["Average", "Good", "Excellent"],
    ordered=True
)

print(df.sort_values("performance"))
```

Benefits:

- Can reduce memory usage
- Allows logical category ordering
- Useful for repeated text values

Part 19: Reading and Writing Files

Pandas supports CSV, Excel, JSON, SQL, Parquet, HTML and several other formats. Reader functions normally start with `read_`, while writer methods start with `to_`. [Official Pandas I/O guide](#)

67. CSV

```
df = pd.read_csv("employees.csv")
```

Useful parameters:

```
df = pd.read_csv(
    "employees.csv",
    usecols=["id", "name", "salary", "joining_date"],
    dtype={"id": "int64", "name": "string"},
    parse_dates=["joining_date"],
    na_values=["NA", "N/A", "unknown"]
)
```


Save:

```
df.to_csv("clean_employees.csv", index=False)
```

68. Excel

```
df = pd.read_excel(  
    "employees.xlsx",  
    sheet_name="Employees"  
)
```

Save:

```
df.to_excel(  
    "result.xlsx",  
    sheet_name="Analysis",  
    index=False  
)
```

Multiple sheets:

```
with pd.ExcelWriter("report.xlsx") as writer:  
    df.to_excel(writer, sheet_name="Employees", index=False)  
    summary.to_excel(writer, sheet_name="Summary", index=False)
```

69. JSON

```
df = pd.read_json("employees.json")  
df.to_json("result.json", orient="records", indent=2)
```

70. Parquet

```
df = pd.read_parquet("employees.parquet")  
df.to_parquet("result.parquet", index=False)
```

Parquet is generally smaller and faster than CSV for analytical data.

71. SQL

```
query = """
SELECT id, name, department, salary
FROM employees
WHERE salary > 50000
"""

df = pd.read_sql(query, connection)
```

Save to database:

```
df.to_sql(
    "employee_result",
    connection,
    if_exists="replace",
    index=False
)
```

Part 20: Visualization

```
import matplotlib.pyplot as plt

department_salary = (
    df.groupby("department")["salary"]
    .mean()
)

department_salary.plot(
    kind="bar",
    color="royalblue",
    title="Average Salary by Department"
)

plt.ylabel("Average Salary")
plt.tight_layout()
plt.show()
```

Other chart types:

```
df["salary"].plot(kind="hist", bins=10)
df.plot(x="experience", y="salary", kind="scatter")
df["department"].value_counts().plot(kind="pie", autopct="%1.1f%%")
```

Part 21: Pandas Performance

72. Prefer Vectorization

Slow approach:

```
annual_salary =  
  
for salary in df["salary"]:  
    annual_salary.append(salary * 12)  
  
df["annual_salary"] = annual_salary
```

Better approach:

```
df["annual_salary"] = df["salary"] * 12
```

73. Avoid **iterrows()** When Possible

Slow:

```
for index, row in df.iterrows():  
    df.loc[index, "bonus"] = row["salary"] * 0.10
```

Better:

```
df["bonus"] = df["salary"] * 0.10
```

74. Reduce Memory

Check memory:

```
print(df.memory_usage(deep=True))  
print(df.memory_usage(deep=True).sum())
```

Convert repeated text to category:

```
df["department"] = df["department"].astype("category")
```

Load only required columns:

```
df = pd.read_csv(
    "large_file.csv",
    usecols=["id", "department", "salary"]
)
```

75. Read Large Files in Chunks

```
total_salary = 0

for chunk in pd.read_csv(
    "large_file.csv",
    chunksize=100000
):
    total_salary += chunk["salary"].sum()

print(total_salary)
```

The Pandas guide recommends reducing loaded columns, using efficient data types, chunking, and more scalable systems when data does not fit into memory. [Scaling guide](#)

76. Copy-on-Write and Safe Assignment

Copy-on-Write is the default in Pandas 3.0. Derived objects behave like independent copies from the user's point of view. Use one clear `.loc` assignment when changing filtered data. [Official Copy-on-Write guide](#)

Avoid chained assignment:

```
# Avoid
df[df["department"] == "AI"]["salary"] = 80000
```

Correct:

```
df.loc[
    df["department"] == "AI",
    "salary"
] = 80000
```

Create an independent working DataFrame when needed:

```
ai_employees = df[  
    df["department"] == "AI"  
].copy()
```

Part 22: Complete Mini Data Analysis Project

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load data
df = pd.read_csv(
    "employees.csv",
    parse_dates=["joining_date"]
)

# 2. Understand data
print(df.head())
print(df.shape)
print(df.info())
print(df.isna().sum())

# 3. Remove duplicate records
df = df.drop_duplicates()

# 4. Clean column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# 5. Clean text
df["name"] = df["name"].str.strip().str.title()
df["department"] = df["department"].str.strip().str.upper()

# 6. Convert salary to numeric
df["salary"] = pd.to_numeric(
    df["salary"],
    errors="coerce"
)

# 7. Fill missing salary by department median
department_median = (
    df.groupby("department")["salary"]
    .transform("median")
)

df["salary"] = df["salary"].fillna(department_median)

# 8. Create new columns
df["annual_salary"] = df["salary"] * 12
df["joining_year"] = df["joining_date"].dt.year
```

```
# 9. Department summary
summary = (
    df.groupby("department", as_index=False)
      .agg(
        employees=("id", "count"),
        average_salary=("salary", "mean"),
        total_salary=("salary", "sum"),
        maximum_salary=("salary", "max")
      )
    .sort_values("average_salary", ascending=False)
)

print(summary)

# 10. Top five employees
top_five = df.nlargest(5, "salary")
print(top_five[["name", "department", "salary"]])

# 11. Save results
df.to_csv("clean_employees.csv", index=False)
summary.to_excel("department_summary.xlsx", index=False)

# 12. Visualize
summary.plot(
    x="department",
    y="average_salary",
    kind="bar",
    legend=False,
    color="steelblue"
)

plt.title("Average Salary by Department")
plt.ylabel("Average Salary")
plt.tight_layout()
plt.show()
```

Part 23: Pandas and SQL Comparison

SQL	Pandas
SELECT name, salary	df[["name", "salary"]]
WHERE salary > 50000	df[df["salary"] > 50000]
ORDER BY salary DESC	df.sort_values("salary", ascending=False)
GROUP BY department	df.groupby("department")

SQL	Pandas
JOIN	<code>df.merge()</code>
COUNT(*)	<code>df.shape[0]</code>
DISTINCT	<code>df.drop_duplicates()</code>
IS NULL	<code>df["salary"].isna()</code>
LIMIT 5	<code>df.head(5)</code>
CASE WHEN	<code>map()</code> , <code>where()</code> or conditional logic

Part 24: Important Interview Questions

1. What is Pandas?

Pandas is a Python library used to read, clean, transform, analyze, and visualize structured data.

2. What is a Series?

A Series is a one-dimensional labeled data structure, similar to one Excel column.

3. What is a DataFrame?

A DataFrame is a two-dimensional table containing rows and columns.

4. Difference between `loc` and `iloc`?

- `loc`: Selects using labels.
- `iloc`: Selects using integer positions.

5. Difference between `merge()` and `concat()`?

- `merge()`: Combines tables using common keys, like SQL JOIN.
- `concat()`: Places tables above or beside each other.

6. Difference between `pivot()` and `pivot_table()`?

- `pivot()` only reshapes and requires unique combinations.
- `pivot_table()` can reshape and aggregate duplicate combinations.

7. Difference between `agg()` and `transform()`?

- `agg()` returns a smaller summary.

- `transform()` returns the same number of rows as the original data.

8. Why is vectorization important?

Vectorized operations work on an entire column and are usually faster and clearer than Python loops.

9. What does `axis=0` mean?

It normally means working down the rows for each column.

10. What does `axis=1` mean?

It normally means working across columns for each row.

11. How do you handle missing values?

Use:

```
isna()
fillna()
dropna()
interpolate()
ffill()
bfill()
```

12. How do you find duplicates?

```
df.duplicated()
```

13. How do you remove duplicates?

```
df.drop_duplicates()
```

14. What is GroupBy?

It splits data into groups, performs calculations on every group, and combines the answers.

15. What is index alignment?

When two Pandas objects are combined, Pandas matches values using index labels rather than only their physical positions.

Recommended Learning Order

- 1. Series and DataFrame
- 2. Reading CSV and Excel
- 3. `head()`, `info()`, `describe()`
- 4. Column selection
- 5. `loc` and `iloc`
- 6. Conditional filtering
- 7. Sorting
- 8. Missing values
- 9. Duplicate handling
- 10. Text and date cleaning
- 11. Aggregation and statistics
- 12. GroupBy
- 13. Merge and concat
- 14. Pivot tables and melt
- 15. Window operations
- 16. MultiIndex and categorical data
- 17. Performance optimization
- 18. Complete EDA projects

This covers the main Pandas concepts required for data analysis, machine learning preprocessing, projects, and technical interviews.

You said:

Recommended Learning Order Series and DataFrame Reading CSV and Excel `head()`, `info()`, `describe()` Column selection `loc` and `iloc` Conditional filtering Sorting Missing values Duplicate handling Text and date cleaning Aggregation and statistics GroupBy Merge and concat Pivot tables and melt Window operations MultiIndex and categorical data Performance optimization Complete EDA projects

ChatGPT said:

Pandas Recommended Learning Order: 30-Day Roadmap

Follow this order because every topic uses concepts learned earlier.

Days	Topic	Important concepts
1–2	Series and DataFrame	Creation, index, columns, data types
3	Reading CSV and Excel	<code>read_csv()</code> , <code>read_excel()</code> , exporting

Days	Topic	Important concepts
4	Understanding data	<code>head()</code> , <code>info()</code> , <code>describe()</code> , <code>shape</code>
5–6	Selecting data	Column selection, <code>loc</code> , <code>iloc</code> , <code>at</code> , <code>iat</code>
7–8	Conditional filtering	Conditions, <code>isin()</code> , <code>between()</code> , <code>query()</code>
9	Sorting	<code>sort_values()</code> , <code>nlargest()</code> , ranking
10–11	Missing values	<code>isna()</code> , <code>fillna()</code> , <code>dropna()</code>
12	Duplicate handling	<code>duplicated()</code> , <code>drop_duplicates()</code>
13–14	Text and date cleaning	<code>.str</code> , <code>to_datetime()</code> , <code>.dt</code>
15–16	Aggregation and statistics	<code>mean()</code> , <code>sum()</code> , <code>agg()</code> , correlation
17–19	GroupBy	Split, apply, combine, <code>transform()</code>
20–21	Merge and concat	SQL joins, combining tables
22–23	Pivot tables and melt	Wide and long data formats
24–25	Window operations	Rolling, expanding, cumulative calculations
26–27	MultilIndex and categories	Hierarchical indexes, categorical data
28	Performance optimization	Vectorization, memory, chunking
29–30	Complete EDA project	Full data cleaning and analysis

Phase 1: Pandas Fundamentals

1. Series and DataFrame — Days 1–2

A **Series** represents one column.

```
import pandas as pd

salary = pd.Series([40000, 50000, 60000])

print(salary)
```

A **DataFrame** represents a complete table.

```
data = {  
    "name": ["Aamir", "Sara", "Ravi"],  
    "department": ["AI", "HR", "AI"],  
    "salary": [60000, 45000, 70000]  
}  
  
df = pd.DataFrame(data)  
  
print(df)
```

Practice:

```
print(df.index)  
print(df.columns)  
print(df.dtypes)  
print(df.shape)
```

Goal: Understand rows, columns, index and data types.

2. Reading CSV and Excel — Day 3

Read CSV:

```
df = pd.read_csv("employees.csv")
```

Read selected columns:

```
df = pd.read_csv(  
    "employees.csv",  
    usecols=["name", "department", "salary"]  
)
```

Read Excel:

```
df = pd.read_excel(  
    "employees.xlsx",  
    sheet_name="Employees"  
)
```

Save results:

```
df.to_csv("clean_data.csv", index=False)
df.to_excel("clean_data.xlsx", index=False)
```

Goal: Load real-world datasets and save processed results.

3. `head()`, `info()` and `describe()` — Day 4

```
print(df.head())      # First 5 rows
print(df.tail())      # Last 5 rows
print(df.sample(3))   # Random 3 rows
print(df.shape)       # Rows and columns
print(df.columns)     # Column names
print(df.dtypes)      # Data types
print(df.info())      # Complete structure
print(df.describe())  # Numeric statistics
print(df.describe(include="all"))
```

Interview point:

- `info()` explains structure, data types and missing-value counts.
- `describe()` provides statistical information.

4. Column Selection, `loc` and `iloc` — Days 5–6

Select one column:

```
print(df["name"])
```

Select multiple columns:

```
print(df[["name", "salary"]])
```

Use `loc` for labels:

```
print(df.loc[0, "name"])
print(df.loc[0:2, ["name", "salary"]])
```

Use `iloc` for positions:

```
print(df.iloc[0, 1])  
print(df.iloc[0:3, 0:2])
```

Single value:

```
print(df.at[0, "name"])  
print(df.iat[0, 1])
```

Goal: Confidently select required rows and columns.

Phase 2: Filtering and Cleaning

5. Conditional Filtering — Days 7–8

Single condition:

```
result = df[df["salary"] > 50000]
```

Multiple conditions:

```
result = df[  
    (df["salary"] > 50000) &  
    (df["department"] == "AI")  
]
```

OR condition:

```
result = df[  
    (df["department"] == "AI") |  
    (df["department"] == "HR")  
]
```

Other important filters:

```
df[df["department"].isin(["AI", "HR"])]  
df[df["salary"].between(45000, 65000)]  
df[df["name"].str.contains("amir", case=False)]  
df.query("salary > 50000")
```

Important: Use **&**, **|** and **~** instead of **and**, **or** and **not** in normal DataFrame conditions.

6. Sorting — Day 9

```
df.sort_values("salary")
```

Descending:

```
df.sort_values("salary", ascending=False)
```

Multiple columns:

```
df.sort_values(  
    ["department", "salary"],  
    ascending=[True, False]  
)
```

Top employees:

```
df.nlargest(3, "salary")  
df.nsmallest(3, "salary")
```

Ranking:

```
df["salary_rank"] = df["salary"].rank(  
    method="dense",  
    ascending=False  
)
```

7. Missing Values — Days 10–11

Find missing values:

```
print(df.isna().sum())
```

Display rows with missing salaries:

```
print(df[df["salary"].isna()])
```

Fill with median:

```
df["salary"] = df["salary"].fillna(  
    df["salary"].median()  
)
```

Fill using department median:

```
group_median = (  
    df.groupby("department")["salary"]  
        .transform("median")  
)  
  
df["salary"] = df["salary"].fillna(group_median)
```

Remove missing rows:

```
df = df.dropna(subset=["salary"])
```

Other methods:

```
df["salary"] = df["salary"].ffill()  
df["salary"] = df["salary"].bfill()  
df["salary"] = df["salary"].interpolate()
```

8. Duplicate Handling — Day 12

Find duplicates:

```
print(df.duplicated())  
print(df.duplicated().sum())
```

Find duplicates based on selected columns:

```
df[df.duplicated(  
    subset=["name", "department"],  
    keep=False  
)]
```

Remove duplicates:


```
df = df.drop_duplicates()
```

Keep the latest occurrence:

```
df = (
    df.sort_values("joining_date")
      .drop_duplicates(
        subset=["name", "department"],
        keep="last"
      )
)
```

9. Text and Date Cleaning — Days 13–14

Clean column names:

```
df.columns = (
    df.columns
      .str.strip()
      .str.lower()
      .str.replace(" ", "_")
)
```

Clean text:

```
df["name"] = df["name"].str.strip().str.title()
df["city"] = df["city"].str.strip().str.lower()
df["department"] = df["department"].str.upper()
```

Convert dates:

```
df["joining_date"] = pd.to_datetime(
    df["joining_date"],
    errors="coerce"
)
```

Extract date information:

```
df["year"] = df["joining_date"].dt.year
df["month"] = df["joining_date"].dt.month
df["month_name"] = df["joining_date"].dt.month_name()
df["day_name"] = df["joining_date"].dt.day_name()
df["quarter"] = df["joining_date"].dt.quarter
```

Phase 3: Data Analysis

10. Aggregation and Statistics — Days 15–16

```
print(df["salary"].count())
print(df["salary"].sum())
print(df["salary"].mean())
print(df["salary"].median())
print(df["salary"].mode())
print(df["salary"].min())
print(df["salary"].max())
print(df["salary"].std())
print(df["salary"].var())
```

Multiple calculations:

```
summary = df["salary"].agg([
    "count",
    "sum",
    "mean",
    "median",
    "min",
    "max"
])

print(summary)
```

Correlation:

```
numeric_data = df.select_dtypes(include="number")
print(numeric_data.corr())
```

11. GroupBy — Days 17–19

Average salary by department:

```
result = (  
    df.groupby("department")["salary"]  
        .mean()  
)  
  
print(result)
```

Multiple aggregations:

```
summary = (  
    df.groupby("department", as_index=False)  
        .agg(  
            employees=("name", "count"),  
            average_salary=("salary", "mean"),  
            total_salary=("salary", "sum"),  
            maximum_salary=("salary", "max")  
        )  
)  
  
print(summary)
```

Group using multiple columns:

```
result = (  
    df.groupby(["department", "city"])["salary"]  
        .mean()  
)  
  
print(result)
```

Use `transform()` when the output must have the same number of rows:

```
df["department_average"] = (  
    df.groupby("department")["salary"]  
        .transform("mean")  
)  
  
print(df["department_average"])
```

Interview difference:

- `agg()` produces a summary.
- `transform()` maintains the original row count.
- `apply()` runs custom logic.

12. Merge and Concat — Days 20–21

Create another table:

```
departments = pd.DataFrame({
    "department": ["AI", "HR", "Sales"],
    "manager": ["Amit", "Priya", "Rahul"]
})
```

Left join:

```
result = df.merge(
    departments,
    on="department",
    how="left"
)
```

Join types:

```
df.merge(departments, how="inner")
df.merge(departments, how="left")
df.merge(departments, how="right")
df.merge(departments, how="outer")
```

Combine rows:

```
combined = pd.concat(
    [january_data, february_data],
    ignore_index=True
)
```

Difference:

- `merge()` combines through a common key.
- `concat()` places tables above or beside each other.

13. Pivot Tables and Melt — Days 22–23

Pivot table:

```
report = pd.pivot_table(  
    df,  
    index="department",  
    columns="city",  
    values="salary",  
    aggfunc="mean",  
    fill_value=0  
)  
  
print(report)
```

Count records:

```
table = pd.crosstab(  
    df["department"],  
    df["city"],  
    margins=True  
)
```

Convert wide data to long data:

```
long_data = df.melt(  
    id_vars=["name"],  
    value_vars=["salary", "experience"],  
    var_name="metric",  
    value_name="value"  
)
```

Interview difference:

- `pivot()` reshapes unique data.
- `pivot_table()` reshapes and aggregates.
- `melt()` converts wide data into long data.

Phase 4: Advanced Pandas

14. Window Operations — Days 24–25

Moving average:

```
df["moving_average"] = (  
    df["salary"]  
        .rolling(window=3, min_periods=1)  
        .mean()  
)
```

Running total:

```
df["running_total"] = df["salary"].cumsum()
```

Previous value:

```
df["previous_salary"] = df["salary"].shift(1)
```

Percentage change:

```
df["salary_change"] = (  
    df["salary"].pct_change() * 100  
)
```

Expanding average:

```
df["expanding_average"] = (  
    df["salary"].expanding().mean()  
)
```

These operations are useful for sales trends, stock prices and time-series analysis.

15. MultiIndex and Categorical Data — Days 26–27

MultiIndex:

```
multi_df = (  
    df.set_index(["department", "city"])  
        .sort_index()  
)  
  
print(multi_df.loc[("AI", "Delhi")])
```

Reset index:

```
normal_df = multi_df.reset_index()
```

Categorical column:

```
df["department"] = df["department"].astype("category")
```

Ordered category:

```
df["performance"] = pd.Categorical(  
    df["performance"],  
    categories=["Low", "Medium", "High"],  
    ordered=True  
)
```

Categorical data can save memory when a column contains repeated values.

16. Performance Optimization — Day 28

Prefer vectorized code:

```
df["annual_salary"] = df["salary"] * 12
```

Avoid unnecessary loops:

```
# Slower  
for index, row in df.iterrows():  
    df.loc[index, "bonus"] = row["salary"] * 0.10  
  
# Faster  
df["bonus"] = df["salary"] * 0.10
```

Read only required columns:

```
df = pd.read_csv(  
    "employees.csv",  
    usecols=["name", "department", "salary"]  
)
```

Process large files in chunks:

```
total = 0

for chunk in pd.read_csv(
    "large_sales.csv",
    chunksize=100000
):
    total += chunk["sales"].sum()

print(total)
```

Check memory:

```
print(df.memory_usage(deep=True))
```

Use safe assignment:

```
df.loc[df["department"] == "AI", "bonus"] = 5000
```

Phase 5: Complete EDA Project — Days 29–30

EDA means Exploratory Data Analysis. Its purpose is to understand the dataset before building reports or machine-learning models.

Project workflow

Load data



Understand rows and columns



Clean column names



Correct data types



Handle missing values



Remove duplicates



Detect outliers



Create new features



Perform GroupBy analysis



Create visualizations



Write conclusions

Complete structure

```
import pandas as pd
import matplotlib.pyplot as plt

# Load
df = pd.read_csv(
    "employees.csv",
    parse_dates=["joining_date"]
)

# Understand
print(df.head())
print(df.shape)
print(df.info())
print(df.describe(include="all"))
print(df.isna().sum())

# Clean columns
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# Remove duplicates
df = df.drop_duplicates()

# Clean text
df["name"] = df["name"].str.strip().str.title()
df["department"] = df["department"].str.strip().str.upper()

# Correct numeric data
df["salary"] = pd.to_numeric(
    df["salary"],
    errors="coerce"
)

# Handle missing salaries
department_median = (
    df.groupby("department")["salary"]
    .transform("median")
)

df["salary"] = df["salary"].fillna(department_median)

# Feature engineering
df["annual_salary"] = df["salary"] * 12
df["joining_year"] = df["joining_date"].dt.year
```

```
# Analysis
summary = (
    df.groupby("department", as_index=False)
      .agg(
          employee_count=("name", "count"),
          average_salary=("salary", "mean"),
          total_salary=("salary", "sum")
      )
      .sort_values("average_salary", ascending=False)
)

print(summary)

# Visualization
summary.plot(
    x="department",
    y="average_salary",
    kind="bar",
    legend=False,
    color="royalblue"
)

plt.title("Average Salary by Department")
plt.ylabel("Average Salary")
plt.tight_layout()
plt.show()

# Export
df.to_csv("clean_employees.csv", index=False)
summary.to_excel("department_report.xlsx", index=False)
```

Final Target

After completing this roadmap, you should be able to:

- Import CSV, Excel, JSON and SQL data.
- Clean missing, duplicate and incorrect values.
- Filter and transform data.
- Perform statistical analysis.
- Analyze groups using `groupby()`.
- Combine multiple tables.
- Build pivot reports.
- Analyze time-series data.
- Optimize larger datasets.

- Complete end-to-end EDA projects.

You said:

Numpy all concept explain with code and basics to Advanced

ChatGPT said:

I'll build this as a complete NumPy learning guide in easy words—from arrays and indexing through broadcasting, linear algebra, random sampling, performance, and interview questions—with runnable code throughout. I'm checking the current official NumPy documentation first.

ChatGPT said:

NumPy: Basics to Advanced with Easy Code

NumPy means **Numerical Python**. It is used for fast calculations involving arrays, matrices, statistics, linear algebra, random numbers, machine learning and scientific computing.

The main NumPy object is `ndarray`, which means **N-dimensional array**. The current official documentation is NumPy 2.5. [Official NumPy beginner guide](#)

Part 1: NumPy Introduction

1. Why Do We Use NumPy?

Python lists are useful, but NumPy arrays provide:

- Faster mathematical calculations
- Less memory usage
- Multi-dimensional arrays
- Matrix operations
- Statistical functions
- Broadcasting
- Linear algebra
- Random data generation
- Machine-learning calculations

Example with a Python list:

```
numbers = [10, 20, 30]

# This repeats the list
print(numbers * 2)
```

Output:

```
[10, 20, 30, 10, 20, 30]
```

NumPy performs mathematical multiplication:

```
import numpy as np

numbers = np.array([10, 20, 30])

print(numbers * 2)
```

Output:

```
[20 40 60]
```

2. Install and Import NumPy

Install:

```
pip install numpy
```

Import:

```
import numpy as np

print(np.__version__)
```

np is the standard short name used for NumPy.

Part 2: NumPy Arrays

3. Creating a NumPy Array

One-dimensional array:

```
import numpy as np

arr = np.array([10, 20, 30, 40])

print(arr)
```

Two-dimensional array:

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(matrix)
```

Three-dimensional array:

```
array_3d = np.array([
    [
        [1, 2],
        [3, 4]
    ],
    [
        [5, 6],
        [7, 8]
    ]
])

print(array_3d)
```

4. Array Dimensions

Dimension	Meaning	Example
0D	Single value	<code>np.array(10)</code>
1D	List or vector	<code>[1, 2, 3]</code>
2D	Table or matrix	<code>[[1, 2], [3, 4]]</code>
3D	Collection of matrices	<code>[[[1, 2]], [[3, 4]]]</code>

Dimension	Meaning	Example
ND	Higher-dimensional data	Images, videos, tensors

```

scalar = np.array(10)
vector = np.array([1, 2, 3])
matrix = np.array([[1, 2], [3, 4]])

print(scalar.ndim) # 0
print(vector.ndim) # 1
print(matrix.ndim) # 2

```

Part 3: Array Properties

5. Important Array Attributes

```

arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(arr.ndim)    # Number of dimensions
print(arr.shape)   # Size of each dimension
print(arr.size)    # Total elements
print(arr.dtype)   # Data type
print(arr.itemsize) # Bytes used by one element
print(arr.nbytes)  # Total memory used

```

Output:

```

2
(2, 3)
6
int64
8
48

```

Important difference:

- **shape**: Arrangement of data
- **size**: Total number of elements

- **ndim**: Number of dimensions

For shape **(2, 3)**:

- 2 rows
- 3 columns
- 6 total elements

Part 4: Array Creation Functions

NumPy provides functions such as **zeros()**, **ones()**, **full()**, **arange()** and **linspace()** for creating arrays.

[Official array-creation guide](#)

6. **zeros()**

```
arr = np.zeros(5)
print(arr)
```

Two-dimensional:

```
arr = np.zeros((2, 3))
print(arr)
```

7. **ones()**

```
arr = np.ones((2, 3))
print(arr)
```

8. **full()**

```
arr = np.full((2, 3), 7)
print(arr)
```

Output:

```
[[7 7 7]
 [7 7 7]]
```


9. `empty()`

```
arr = np.empty((2, 3))  
print(arr)
```

`empty()` creates an array without initializing its values. Its existing values are unpredictable, so fill it before using it.

10. `arange()`

Similar to Python's `range()`:

```
arr = np.arange(1, 10)  
  
print(arr)
```

With step:

```
arr = np.arange(0, 20, 2)  
  
print(arr)
```

Output:

```
[ 0  2  4  6  8 10 12 14 16 18]
```

11. `linspace()`

Creates equally spaced values:

```
arr = np.linspace(0, 10, 5)  
  
print(arr)
```

Output:

```
[ 0.  2.5  5.  7.5 10.]
```

Difference:

- `arange()` controls the step.

- `linspace()` controls the number of values.

12. Identity Matrix

```
identity = np.eye(3)

print(identity)
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

13. Diagonal Matrix

```
diagonal = np.diag([10, 20, 30])

print(diagonal)
```

Extract a diagonal:

```
matrix = np.array([
    [10, 20],
    [30, 40]
])

print(np.diag(matrix))
```

14. Create from a Function

```
arr = np.fromfunction(
    lambda row, column: row + column,
    (3, 3)
)

print(arr)
```

Part 5: NumPy Data Types

15. Common NumPy Data Types

Data type	Meaning
<code>int8</code> , <code>int16</code> , <code>int32</code> , <code>int64</code>	Integer
<code>uint8</code>	Unsigned positive integer
<code>float32</code> , <code>float64</code>	Decimal number
<code>bool</code>	True or False
<code>complex64</code> , <code>complex128</code>	Complex number
<code>str_</code>	String
<code>datetime64</code>	Date and time
<code>timedelta64</code>	Time difference

Specify data type:

```
arr = np.array([10, 20, 30], dtype=np.int32)

print(arr.dtype)
```

16. Convert Data Type with `astype()`

```
arr = np.array([10.5, 20.7, 30.9])

integer_arr = arr.astype(np.int32)

print(integer_arr)
```

Output:

```
[10 20 30]
```

Convert integers to floats:

```
arr = np.array([1, 2, 3])

float_arr = arr.astype(np.float64)

print(float_arr)
```

17. Data Type Overflow

Small data types have limited ranges.

```
arr = np.array([127], dtype=np.int8)

print(arr.dtype)
```

`int8` supports only `-128` through `127`. Select a sufficiently large data type for your values.

Part 6: Indexing and Slicing

NumPy indexing starts from zero and supports normal indexing, negative indexing, slicing, Boolean indexing and advanced indexing. [Official indexing guide](#)

18. One-Dimensional Indexing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0]) # 10
print(arr[2]) # 30
print(arr[-1]) # 50
```

Update an element:

```
arr[1] = 200

print(arr)
```

19. One-Dimensional Slicing

Syntax:

```
array[start:stop:step]
```

```
arr = np.array([10, 20, 30, 40, 50, 60])

print(arr[1:4])
print(arr[:3])
print(arr[3:])
print(arr[:2])
print(arr[::2])
print(arr[::-1])
```

Remember: The stop position is not included.

20. Two-Dimensional Indexing

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print(matrix[0, 0]) # 10
print(matrix[1, 2]) # 60
print(matrix[-1, -1]) # 90
```

Select a complete row:

```
print(matrix[1, :])
```

Select a complete column:

```
print(matrix[:, 1])
```

Select a submatrix:

```
print(matrix[0:2, 1:3])
```

Output:

```
[[20 30]
 [50 60]]
```

21. Boolean Indexing

```
arr = np.array([10, 20, 30, 40, 50])  
  
result = arr[arr > 25]  
  
print(result)
```

Output:

```
[30 40 50]
```

Multiple conditions:

```
result = arr[  
    (arr >= 20) &  
    (arr <= 40)  
]  
  
print(result)
```

Use:

- **&**: AND
- **|**: OR
- **~**: NOT

22. Fancy or Advanced Indexing

Select elements using positions:

```
arr = np.array([10, 20, 30, 40, 50])  
  
positions = [0, 2, 4]  
  
print(arr[positions])
```

Select rows:

```
matrix = np.array([
    [10, 20],
    [30, 40],
    [50, 60]
])

print(matrix[[0, 2]])
```

23. `where()`

Return values based on a condition:

```
arr = np.array([10, 20, 30, 40])

result = np.where(
    arr >= 30,
    "High",
    "Low"
)

print(result)
```

Replace negative values with zero:

```
arr = np.array([-5, 10, -2, 20])

clean = np.where(arr < 0, 0, arr)

print(clean)
```

24. `select()`

Used for multiple conditions:

```
marks = np.array([35, 55, 75, 90])

conditions = [
    marks >= 80,
    marks >= 60,
    marks >= 40
]

choices = ["Excellent", "Good", "Pass"]

result = np.select(
    conditions,
    choices,
    default="Fail"
)

print(result)
```

Part 7: Shape Manipulation

25. `reshape()`

```
arr = np.arange(1, 13)

matrix = arr.reshape(3, 4)

print(matrix)
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Automatic dimension:

```
matrix = arr.reshape(4, -1)

print(matrix.shape)
```

NumPy automatically calculates the `-1` dimension.

26. Flatten an Array

Using `flatten()`:

```
matrix = np.array([
    [1, 2],
    [3, 4]
])

flat = matrix.flatten()
```

Using `ravel()`:

```
flat = matrix.ravel()
```

Difference:

- `flatten()` always returns a copy.
- `ravel()` returns a view when possible.
- `reshape(-1)` is another common option.

27. Transpose

Convert rows into columns:

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix.T)
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Another form:

```
print(np.transpose(matrix))
```

28. Add and Remove Dimensions

Add dimension:

```
arr = np.array([1, 2, 3])

row = arr[np.newaxis, :]
column = arr[:, np.newaxis]

print(row.shape)  # (1, 3)
print(column.shape) # (3, 1)
```

Using `expand_dims()`:

```
expanded = np.expand_dims(arr, axis=0)
```

Remove dimensions of size one:

```
arr = np.array([[[[1, 2, 3]]]])

result = np.squeeze(arr)

print(result.shape)
```

29. Swap Axes

```
arr = np.arange(24).reshape(2, 3, 4)

swapped = np.swapaxes(arr, 0, 2)

print(swapped.shape)
```

Part 8: Combining and Splitting Arrays

30. `concatenate()`

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = np.concatenate([a, b])

print(result)
```

For matrices:

```
a = np.array([[1, 2]])
b = np.array([[3, 4]])

rows = np.concatenate([a, b], axis=0)
columns = np.concatenate([a, b], axis=1)
```

31. `vstack()` and `hstack()`

Vertical stacking:

```
a = np.array([1, 2])
b = np.array([3, 4])

print(np.vstack([a, b]))
```

Horizontal stacking:

```
print(np.hstack([a, b]))
```

32. `stack()`

Creates a new dimension:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = np.stack([a, b], axis=0)

print(result)
```

33. Split Arrays

```
arr = np.arange(12)

parts = np.split(arr, 3)

print(parts)
```

Unequal division:

```
parts = np.array_split(arr, 5)

print(parts)
```

Matrix splitting:

```
matrix = np.arange(16).reshape(4, 4)

print(np.vsplit(matrix, 2))
print(np.hsplit(matrix, 2))
```

Part 9: Mathematical Operations

34. Scalar Operations

```
arr = np.array([10, 20, 30])

print(arr + 5)
print(arr - 5)
print(arr * 2)
print(arr / 2)
print(arr ** 2)
print(arr % 3)
```

35. Array-to-Array Operations

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

These are element-by-element operations.

36. Universal Functions

A universal function, or **ufunc**, operates element by element and supports broadcasting and type handling. [Official ufunc guide](#)

```
arr = np.array([1, 4, 9, 16])

print(np.sqrt(arr))
print(np.square(arr))
print(np.exp(arr))
print(np.log(arr))
print(np.abs(arr))
```

Trigonometric functions:

```
angles = np.array([0, 30, 45, 60, 90])

radians = np.deg2rad(angles)

print(np.sin(radians))
print(np.cos(radians))
print(np.tan(radians))
```

Rounding:

```
arr = np.array([1.234, 2.567, 3.891])

print(np.round(arr, 2))
print(np.floor(arr))
print(np.ceil(arr))
print(np.trunc(arr))
```

37. Comparison Operations

```
a = np.array([10, 20, 30])
b = np.array([10, 15, 35])

print(a == b)
print(a > b)
print(a < b)
```

Check all values:

```
print(np.all(a > 0))
```

Check whether any value matches:

```
print(np.any(a > 25))
```

Compare floating-point values:

```
x = np.array([0.1 + 0.2])
y = np.array([0.3])

print(np.isclose(x, y))
print(np.allclose(x, y))
```

Do not always use `==` for decimal calculations because floating-point values can have tiny precision differences.

Part 10: Axis Concept

38. Understanding `axis=0` and `axis=1`

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

Sum all values:

```
print(np.sum(matrix))
```

Column-wise result:

```
print(np.sum(matrix, axis=0))
```

Output:

```
[5 7 9]
```

Row-wise result:

```
print(np.sum(matrix, axis=1))
```

Output:

```
[ 6 15]
```

Easy memory trick:

- **axis=0**: Collapse rows and get one answer per column.
- **axis=1**: Collapse columns and get one answer per row.

Keep the removed dimension:

```
result = np.sum(  
    matrix,  
    axis=1,  
    keepdims=True  
)  
  
print(result.shape)
```

Part 11: Aggregation and Statistics

39. Basic Aggregations

```
arr = np.array([10, 20, 30, 40, 50])

print(np.sum(arr))
print(np.mean(arr))
print(np.median(arr))
print(np.min(arr))
print(np.max(arr))
print(np.std(arr))
print(np.var(arr))
print(np.prod(arr))
print(np.ptp(arr))    # Maximum - minimum
```

40. Position of Minimum and Maximum

```
arr = np.array([40, 10, 70, 20])

print(np.argmin(arr)) # Position of minimum
print(np.argmax(arr)) # Position of maximum
```

41. Percentile and Quantile

```
arr = np.array([10, 20, 30, 40, 50])

print(np.percentile(arr, 25))
print(np.percentile(arr, 50))
print(np.percentile(arr, 75))

print(np.quantile(arr, 0.25))
print(np.quantile(arr, 0.50))
print(np.quantile(arr, 0.75))
```


42. Weighted Average

```
marks = np.array([80, 70, 90])
weights = np.array([0.3, 0.2, 0.5])

weighted_average = np.average(
    marks,
    weights=weights
)

print(weighted_average)
```

Formula:

$$\text{Weighted average} = \frac{\sum (x_i w_i)}{\sum w_i}$$

43. Cumulative Operations

```
arr = np.array([10, 20, 30, 40])

print(np.cumsum(arr))
print(np.cumprod(arr))
```

Output for cumulative sum:

```
[ 10  30  60 100]
```

Difference between consecutive values:

```
print(np.diff(arr))
```

Output:

```
[10 10 10]
```

44. Correlation and Covariance

```
experience = np.array([1, 2, 3, 4, 5])
salary = np.array([30000, 40000, 50000, 60000, 70000])

correlation = np.corrcoef(experience, salary)
covariance = np.cov(experience, salary)

print(correlation)
print(covariance)
```

Part 12: Broadcasting

Broadcasting allows NumPy to calculate with arrays having different but compatible shapes, usually without creating unnecessary copies. [Official broadcasting guide](#)

45. Basic Broadcasting

```
arr = np.array([10, 20, 30])

print(arr + 5)
```

NumPy behaves as if **5** were:

```
[5, 5, 5]
```

46. Matrix and Row Broadcasting

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

row = np.array([1, 2, 3])

print(matrix + row)
```

Output:

```
[[11 22 33]
 [41 52 63]]
```

Shapes:

```
Matrix: (2, 3)
Row:    (3,)
```

They are compatible because their last dimensions match.

47. Column Broadcasting

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

column = np.array([
    [100],
    [200]
])

print(matrix + column)
```

48. Broadcasting Rules

Compare shapes from the right side.

Dimensions are compatible when:

1. They are equal, or
2. One of them is **1**, or
3. One shape has no dimension at that position.

Compatible:

```
(2, 3)
(3)
```

Compatible:

```
(2, 3)
(2, 1)
```

Not compatible:

```
(2, 3)
(2)
```

Part 13: Sorting, Searching and Set Operations

49. Sorting

```
arr = np.array([40, 10, 30, 20])

sorted_arr = np.sort(arr)

print(sorted_arr)
```

Descending:

```
descending = np.sort(arr)[::-1]
```

Sort a matrix:

```
matrix = np.array([
    [30, 10],
    [40, 20]
])

print(np.sort(matrix, axis=0))
print(np.sort(matrix, axis=1))
```

50. **argsort()**

Returns the positions that would sort the array:

```
arr = np.array([40, 10, 30, 20])

positions = np.argsort(arr)

print(positions)
print(arr[positions])
```

51. searchsorted()

Find where a value should be inserted in sorted data:

```
arr = np.array([10, 20, 30, 40])

position = np.searchsorted(arr, 25)

print(position)
```

52. Unique Values

```
arr = np.array([10, 20, 10, 30, 20, 10])

values, counts = np.unique(
    arr,
    return_counts=True
)

print(values)
print(counts)
```

53. Set Operations

```
a = np.array([1, 2, 3, 4])
b = np.array([3, 4, 5, 6])

print(np.union1d(a, b))
print(np.intersect1d(a, b))
print(np.setdiff1d(a, b))
print(np.setxor1d(a, b))
print(np.isin(a, b))
```

Part 14: Missing and Infinite Values

54. Missing Values with NaN

```
arr = np.array([10, 20, np.nan, 40])

print(np.isnan(arr))
```

Remove missing values:

```
clean = arr[~np.isnan(arr)]

print(clean)
```

Replace missing values:

```
filled = np.nan_to_num(
    arr,
    nan=0
)

print(filled)
```

55. NaN-Aware Calculations

Normal mean:

```
print(np.mean(arr))
```

Output:

```
nan
```

Ignore missing values:

```
print(np.nanmean(arr))
print(np.nansum(arr))
print(np.nanmin(arr))
print(np.nanmax(arr))
print(np.nanmedian(arr))
print(np.nanstd(arr))
```

56. Infinite Values

```
arr = np.array([10, np.inf, -np.inf, 40])

print(np.isinf(arr))
print(np.isfinite(arr))
```

Replace infinite values:

```
clean = np.nan_to_num(
    arr,
    posinf=999,
    neginf=-999
)

print(clean)
```

Important: **NaN** is normally used with floating-point arrays, not ordinary integer arrays.

Part 15: Random Numbers

Use NumPy's modern **Generator** API:

```
rng = np.random.default_rng(seed=42)
```

The seed makes results reproducible.

57. Random Decimal Values

```
values = rng.random(5)

print(values)
```

Matrix:

```
matrix = rng.random((2, 3))
```

58. Random Integers

```
numbers = rng.integers(  
    low=1,  
    high=11,  
    size=5  
)  
  
print(numbers)
```

high=11 means 11 is not included.

59. Normal Distribution

```
data = rng.normal(  
    loc=100,    # Mean  
    scale=15,   # Standard deviation  
    size=1000  
)  
  
print(data.mean())  
print(data.std())
```

60. Random Choice

```
colors = np.array(["Red", "Blue", "Green"])  
  
sample = rng.choice(  
    colors,  
    size=5,  
    replace=True  
)  
  
print(sample)
```

Without replacement:


```
sample = rng.choice(  
    colors,  
    size=3,  
    replace=False  
)
```

With probabilities:

```
sample = rng.choice(  
    colors,  
    size=10,  
    p=[0.5, 0.3, 0.2]  
)
```

61. Shuffle and Permutation

shuffle() changes the original array:

```
arr = np.array([1, 2, 3, 4, 5])  
  
rng.shuffle(arr)  
  
print(arr)
```

permutation() returns a shuffled copy:

```
arr = np.array([1, 2, 3, 4, 5])  
  
shuffled = rng.permutation(arr)  
  
print(arr)  
print(shuffled)
```

Part 16: Matrix and Linear Algebra

62. Element-Wise vs Matrix Multiplication

```
a = np.array([
    [1, 2],
    [3, 4]
])

b = np.array([
    [5, 6],
    [7, 8]
])
```

Element-wise multiplication:

```
print(a * b)
```

Matrix multiplication:

```
print(a @ b)
```

Other forms:

```
print(np.matmul(a, b))
print(np.dot(a, b))
```

For two-dimensional matrices, `@`, `matmul()` and `dot()` give matrix multiplication. Prefer `@` for clarity.

63. Matrix Transpose

```
print(a.T)
```

64. Matrix Determinant

```
determinant = np.linalg.det(a)

print(determinant)
```

65. Matrix Inverse

```
inverse = np.linalg.inv(a)

print(inverse)
```

Check:

```
print(a @ inverse)
```

The result should be close to an identity matrix.

66. Solve Linear Equations

Equations:

[$3x+y=9$]

[$x+2y=8$]

```
A = np.array([
    [3, 1],
    [1, 2]
], dtype=float)

b = np.array([9, 8], dtype=float)

solution = np.linalg.solve(A, b)

print(solution)
```

Output:

```
[2. 3.]
```

Prefer `solve(A, b)` over calculating `inv(A) @ b`. It is generally faster and numerically safer.

67. Matrix Rank

```
rank = np.linalg.matrix_rank(A)

print(rank)
```

68. Eigenvalues and Eigenvectors

```
matrix = np.array([
    [2, 1],
    [1, 2]
], dtype=float)

eigenvalues, eigenvectors = np.linalg.eigh(matrix)

print(eigenvalues)
print(eigenvectors)
```

`eigh()` is suitable for real symmetric matrices.

69. Singular Value Decomposition

```
U, singular_values, Vt = np.linalg.svd(A)

print(U)
print(singular_values)
print(Vt)
```

SVD is used in:

- PCA
- Image compression
- Recommendation systems
- Dimensionality reduction
- Noise reduction

70. Vector Norm

```
vector = np.array([3, 4])

length = np.linalg.norm(vector)

print(length)
```

Output:

```
5.0
```

71. Least-Squares Solution

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])

X = np.column_stack([
    np.ones(len(x)),
    x
])

coefficients = np.linalg.lstsq(
    X,
    y,
    rcond=None
)[0]

intercept, slope = coefficients

print(intercept)
print(slope)
```

This finds the best-fitting line:

[$y=b_0+b_1x$]

Part 17: Copy and View

A view shares the original array's memory, while a copy owns separate data. Basic slicing normally creates a view; advanced indexing creates a copy. [Official copies-and-views guide](#)

72. Assignment Shares the Same Array

```
original = np.array([10, 20, 30])

another = original

another[0] = 999

print(original)
```

Output:

```
[999 20 30]
```

Both names point to the same array.

73. View

```
original = np.array([10, 20, 30, 40])  
  
view = original[1:3]  
  
view[0] = 999  
  
print(original)
```

Output:

```
[ 10 999  30  40]
```

74. Copy

```
original = np.array([10, 20, 30, 40])  
  
copied = original.copy()  
  
copied[0] = 999  
  
print(original)  
print(copied)
```

Changes to **copied** do not affect **original**.

Check memory sharing:

```
print(np.shares_memory(original, copied))
```

Part 18: Input and Output

75. Save and Load NumPy Arrays

Save one array:

```
arr = np.array([10, 20, 30])  
  
np.save("numbers.npy", arr)
```

Load it:

```
loaded = np.load("numbers.npy")  
  
print(loaded)
```

76. Save Multiple Arrays

```
x = np.array([1, 2, 3])  
y = np.array([10, 20, 30])  
  
np.savez(  
    "arrays.npz",  
    features=x,  
    target=y  
)
```

Load:

```
data = np.load("arrays.npz")  
  
print(data["features"])  
print(data["target"])
```

Compressed:

```
np.savez_compressed(  
    "compressed_arrays.npz",  
    features=x,  
    target=y  
)
```

77. Text and CSV Files

Save:

```
matrix = np.array([
    [10, 20],
    [30, 40]
])

np.savetxt(
    "numbers.csv",
    matrix,
    delimiter=",",
    fmt="%d"
)
```

Load:

```
loaded = np.loadtxt(
    "numbers.csv",
    delimiter=",",
)
```

Handle missing values:

```
loaded = np.genfromtxt(
    "numbers.csv",
    delimiter=",",
    skip_header=1,
    missing_values="",
    filling_values=np.nan
)
```

Part 19: Advanced NumPy Concepts

78. Ufunc Methods

Reduce:

```
arr = np.array([1, 2, 3, 4])

print(np.add.reduce(arr))
```

Equivalent to:


```
print(np.sum(arr))
```

Accumulate:

```
print(np.add.accumulate(arr))
```

Output:

```
[ 1  3  6 10]
```

Outer operation:

```
a = np.array([1, 2, 3])
b = np.array([10, 20])

print(np.multiply.outer(a, b))
```

79. einsum()

Matrix multiplication:

```
a = np.array([
    [1, 2],
    [3, 4]
])

b = np.array([
    [5, 6],
    [7, 8]
])

result = np.einsum(
    "ij,jk->ik",
    a,
    b
)

print(result)
```

einsum() is powerful for tensor and multi-dimensional calculations.

80. Meshgrid

```
x = np.array([1, 2, 3])
y = np.array([10, 20])

X, Y = np.meshgrid(x, y)

print(X)
print(Y)
```

Used for:

- 3D surfaces
- Contour plots
- Coordinate grids
- Mathematical simulations

81. Structured Arrays

A structured array can contain different data types in one record.

```
employee_type = np.dtype([
    ("name", "U20"),
    ("age", "i4"),
    ("salary", "f8")
])

employees = np.array([
    ("Aamir", 24, 60000),
    ("Sara", 25, 55000)
], dtype=employee_type)

print(employees["name"])
print(employees["salary"])
```

For normal business analysis, Pandas DataFrames are generally easier than structured arrays.

82. Masked Arrays

```
data = np.array([10, -999, 30, 40])

masked = np.ma.masked_where(
    data == -999,
    data
)

print(masked)
print(masked.mean())
```

The invalid value is ignored in the calculation.

83. Memory-Mapped Arrays

Useful when the array is too large to load completely into RAM:

```
large_array = np.memmap(
    "large_data.dat",
    dtype="float32",
    mode="w+",
    shape=(10000, 1000)
)

large_array[0, 0] = 100
large_array.flush()
```

84. Fast Fourier Transform

```
signal = np.array([0, 1, 0, -1])

frequencies = np.fft.fft(signal)

print(frequencies)
```

FFT is used in:

- Audio processing
- Signal analysis
- Image processing

- Frequency detection

85. Polynomial Fitting

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 5, 10, 17, 26])

model = np.polynomial.Polynomial.fit(
    x,
    y,
    deg=2
)

predictions = model(x)

print(predictions)
```

Part 20: NumPy for Machine Learning

86. Feature Standardization

Formula:

$$[z = \frac{x - \mu}{\sigma}]$$

```
features = np.array([10, 20, 30, 40, 50])

standardized = (
    features - features.mean()
) / features.std()

print(standardized)
```

After standardization:

- Mean is approximately 0
- Standard deviation is approximately 1

87. Min-Max Normalization

Formula:

$$[x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}]$$

```
normalized = (  
    features - features.min()  
) / (  
    features.max() - features.min()  
)  
  
print(normalized)
```

Values are converted between 0 and 1.

88. Mean Squared Error

Formula:

[$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$]

```
actual = np.array([100, 200, 300])  
predicted = np.array([110, 190, 310])  
  
mse = np.mean(  
    (actual - predicted) ** 2  
)  
  
print(mse)
```

89. Mean Absolute Error

```
mae = np.mean(  
    np.abs(actual - predicted)  
)  
  
print(mae)
```

90. One-Hot Encoding

```
labels = np.array([0, 2, 1, 2])  
  
number_of_classes = 3  
  
one_hot = np.eye(number_of_classes)[labels]  
  
print(one_hot)
```

91. ReLU Activation

Formula:

$$[\text{ReLU}(x) = \max(0, x)]$$

```
values = np.array([-5, -1, 0, 3, 8])  
  
relu = np.maximum(0, values)  
  
print(relu)
```

92. Sigmoid Activation

Formula:

$$[\sigma(x) = \frac{1}{1 + e^{-x}}]$$

```
values = np.array([-2, -1, 0, 1, 2])  
  
sigmoid = 1 / (1 + np.exp(-values))  
  
print(sigmoid)
```

93. Softmax Activation

```
scores = np.array([2.0, 1.0, 0.1])  
  
shifted_scores = scores - np.max(scores)  
  
probabilities = (  
    np.exp(shifted_scores) /  
    np.sum(np.exp(shifted_scores))  
)  
  
print(probabilities)  
print(probabilities.sum())
```

Subtracting the maximum improves numerical stability.

Part 21: Performance Optimization

94. Prefer Vectorization

Slow Python loop:

```
arr = np.arange(1_000_000)

result =

for value in arr:
    result.append(value * 2)
```

Fast NumPy operation:

```
result = arr * 2
```

95. Avoid `np.vectorize()` for Speed

```
def square(value):
    return value ** 2

vectorized_square = np.vectorize(square)

result = vectorized_square(arr)
```

`np.vectorize()` makes code convenient but generally does not provide true NumPy-level speed. Prefer:

```
result = arr ** 2
```

96. Preallocate Arrays

Slower repeated appending:

```
result = np.array()

for value in range(1000):
    result = np.append(result, value)
```

Better:

```
result = np.empty(1000, dtype=np.int64)

for value in range(1000):
    result[value] = value
```

Best when possible:

```
result = np.arange(1000)
```

97. Use Suitable Data Types

```
large = np.arange(
    1_000_000,
    dtype=np.int64
)

small = np.arange(
    1_000_000,
    dtype=np.int32
)

print(large.nbytes)
print(small.nbytes)
```

`int32` normally uses half the memory of `int64`.

98. In-Place Operations

```
arr = np.array([1.0, 2.0, 3.0])

arr *= 2

print(arr)
```

In-place operations can reduce temporary arrays, but ensure the existing data type can hold the result.

Part 22: Crop Yield Mini Project

```
import numpy as np

# Columns:
# rainfall, temperature, fertilizer, crop_yield
data = np.array([
    [800, 25, 100, 3.2],
    [950, 24, 120, 3.8],
    [700, 29, 90, 2.7],
    [1100, 23, 140, 4.5],
    [650, 31, 80, 2.4],
    [1000, 26, 130, 4.0]
], dtype=float)

# Separate features and target
X = data[:, :3]
y = data[:, 3]

print("Feature shape:", X.shape)
print("Target shape:", y.shape)

# Basic statistics
print("Feature means:", X.mean(axis=0))
print("Feature minimums:", X.min(axis=0))
print("Feature maximums:", X.max(axis=0))
print("Average crop yield:", y.mean())

# Find high-yield records
high_yield = data[y >= 4.0]

print("High-yield records:")
print(high_yield)

# Standardize features
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)

X_scaled = (X - X_mean) / X_std

# Add intercept column
design_matrix = np.column_stack([
    np.ones(X_scaled.shape[0]),
    X_scaled
])

# Train a simple linear regression model
weights = np.linalg.lstsq(
    design_matrix,
    y,
```

```

    rcond=None
)[0]

# Predict training values
predictions = design_matrix @ weights

# Calculate error
mse = np.mean((y - predictions) ** 2)

print("Weights:", weights)
print("Predictions:", predictions)
print("MSE:", mse)

# Predict crop yield for new data
new_farm = np.array([
    900, 27, 110
], dtype=float)

new_farm_scaled = (
    new_farm - X_mean
) / X_std

new_design = np.column_stack([
    np.ones(new_farm_scaled.shape[0]),
    new_farm_scaled
])

predicted_yield = new_design @ weights

print("Predicted crop yield:", predicted_yield)

```

Part 23: Important Interview Questions

1. What is NumPy?

NumPy is a Python library used for fast numerical calculations with arrays and matrices.

2. What is **ndarray**?

ndarray is NumPy's N-dimensional array object.

3. NumPy array vs Python list?

NumPy array	Python list
Faster numerical operations	Slower for large calculations

NumPy array	Python list
Usually one data type	Can contain mixed data types
Supports broadcasting	Does not support broadcasting
Uses less memory for numeric data	Usually uses more memory
Supports matrix operations	Requires loops or extra code

4. What is vectorization?

Vectorization means applying an operation to a complete array without writing a Python loop.

```
result = arr * 2
```

5. What is broadcasting?

Broadcasting allows operations between arrays with different but compatible shapes.

6. Difference between **shape**, **size** and **ndim**?

- **shape**: Size of every dimension
- **size**: Total number of elements
- **ndim**: Number of dimensions

7. Difference between **flatten()** and **ravel()**?

- **flatten()** always creates a copy.
- **ravel()** returns a view when possible.

8. Difference between copy and view?

- Copy has independent data.
- View shares memory with the original array.

9. Difference between ***** and **@**?

- *****: Element-wise multiplication
- **@**: Matrix multiplication

10. What does **axis=0** mean?

It performs the operation down the rows and produces one result per column.

11. What does **axis=1** mean?

It performs the operation across the columns and produces one result per row.

12. Difference between **arange()** and **linspace()**?

- **arange()** specifies the step.
- **linspace()** specifies the number of values.

13. Why is NumPy faster?

NumPy uses compact homogeneous arrays and performs many operations in optimized compiled code instead of ordinary Python loops.

14. How do you handle missing values?

```
np.isnan()
np.nanmean()
np.nansum()
np.nan_to_num()
```

15. Why use **default_rng()**?

It is NumPy's modern random-number interface and supports reproducible, independent random generators.

Recommended NumPy Learning Order

1. Installation and import
2. NumPy arrays and dimensions
3. Array properties
4. Array-creation functions
5. Data types and conversion
6. Indexing and slicing
7. Boolean and fancy indexing
8. Reshaping and transpose
9. Combining and splitting arrays
10. Mathematical operations
11. Axis concept
12. Aggregation and statistics
13. Sorting and searching
14. Missing values
15. Broadcasting

- 16. Random-number generation
- 17. Matrix operations
- 18. Linear algebra
- 19. Copy and view
- 20. File input and output
- 21. Vectorization and performance
- 22. Advanced ufuncs and einsum()
- 23. NumPy for machine learning
- 24. Complete numerical projects

You said:

Recommended NumPy Learning Order Installation and import NumPy arrays and dimensions Array properties Array-creation functions Data types and conversion Indexing and slicing Boolean and fancy indexing Reshaping and transpose Combining and splitting arrays Mathematical operations Axis concept Aggregation and statistics Sorting and searching Missing values Broadcasting Random-number generation Matrix operations Linear algebra Copy and view File input and output Vectorization and performance Advanced ufuncs and einsum() NumPy for machine learning Complete numerical projects

ChatGPT said:

Recommended NumPy Learning Order: 30-Day Roadmap

Follow this order because advanced NumPy concepts such as broadcasting, linear algebra and machine learning depend on strong array fundamentals.

Days	Topic	Learning outcome
1	Installation and import	Set up NumPy
2	Arrays and dimensions	Create 0D, 1D, 2D and 3D arrays
3	Array properties	Understand shape, size, dtype and memory
4	Array-creation functions	Create arrays efficiently
5	Data types and conversion	Control memory and numeric types
6–7	Indexing and slicing	Select and update array data
8	Boolean and fancy indexing	Filter using conditions and positions
9	Reshaping and transpose	Change array structure

Days	Topic	Learning outcome
10	Combining and splitting	Join and divide arrays
11	Mathematical operations	Perform vectorized calculations
12	Axis concept	Calculate row-wise and column-wise
13–14	Aggregation and statistics	Analyze numerical data
15	Sorting and searching	Sort values and locate elements
16	Missing values	Handle NaN and infinity
17–18	Broadcasting	Calculate with different array shapes
19	Random numbers	Generate samples and simulations
20	Matrix operations	Perform matrix calculations
21	Linear algebra	Solve equations and decompositions
22	Copy and view	Understand NumPy memory behavior
23	File input and output	Save and load arrays
24–25	Vectorization and performance	Write faster numerical code
26	Advanced ufuncs and einsum()	Perform advanced array calculations
27–28	NumPy for machine learning	Preprocess data and implement formulas
29–30	Complete numerical projects	Build end-to-end NumPy projects

Phase 1: NumPy Fundamentals

1. Installation and Import — Day 1

Install NumPy:

```
pip install numpy
```

Import:

```
import numpy as np

print(np.__version__)
```

Initial practice:

```
numbers = np.array([10, 20, 30])

print(numbers)
print(type(numbers))
```

Goal: Understand that `ndarray` is NumPy's main array object.

2. NumPy Arrays and Dimensions — Day 2

Zero-dimensional array

```
scalar = np.array(10)

print(scalar)
print(scalar.ndim)
```

One-dimensional array

```
vector = np.array([10, 20, 30])

print(vector.ndim)
```

Two-dimensional array

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(matrix.ndim)
```

Three-dimensional array

```
tensor = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(tensor.ndim)
```

Easy meaning:

- 0D: One value
- 1D: Vector
- 2D: Matrix or table
- 3D: Collection of matrices
- ND: Higher-dimensional tensor

3. Array Properties — Day 3

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(arr.ndim)    # Dimensions
print(arr.shape)   # Rows and columns
print(arr.size)    # Total elements
print(arr.dtype)   # Data type
print(arr.itemsize) # Memory per element
print(arr.nbytes)  # Total memory
```

For this array:

```
shape = (2, 3)
size  = 6
ndim  = 2
```

Practice question: Create an array with shape (3, 4) and print all its properties.

4. Array-Creation Functions — Day 4

```
print(np.zeros((2, 3)))  
print(np.ones((2, 3)))  
print(np.full((2, 3), 7))  
print(np.eye(3))
```

Using `arange()`:

```
numbers = np.arange(0, 20, 2)  
  
print(numbers)
```

Using `linspace()`:

```
numbers = np.linspace(0, 10, 5)  
  
print(numbers)
```

Difference:

- `arange()` controls the step.
- `linspace()` controls the number of values.

Other functions:

```
print(np.diag([10, 20, 30]))  
print(np.empty((2, 3)))
```

Practice: Create numbers from 10 to 100 with a step of 10 and reshape them into a matrix.

5. Data Types and Conversion — Day 5

```
arr = np.array(  
    [10, 20, 30],  
    dtype=np.int32  
)  
  
print(arr.dtype)
```

Convert data type:

```
decimal_values = np.array([10.5, 20.8, 30.2])

integer_values = decimal_values.astype(np.int32)

print(integer_values)
```

Common data types:

Type	Use
<code>int32</code> , <code>int64</code>	Integers
<code>float32</code> , <code>float64</code>	Decimal values
<code>bool</code>	True or False
<code>complex128</code>	Complex numbers
<code>datetime64</code>	Dates

Memory comparison:

```
a = np.arange(1_000_000, dtype=np.int64)
b = np.arange(1_000_000, dtype=np.int32)

print(a.nbytes)
print(b.nbytes)
```

Phase 2: Selecting and Organizing Arrays

6. Indexing and Slicing — Days 6–7

One-dimensional indexing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])
print(arr[-1])
print(arr[1:4])
print(arr[:2])
print(arr[:-1])
```

Two-dimensional indexing

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print(matrix[1, 2])  # 60
print(matrix[1, :])  # Second row
print(matrix[:, 1])  # Second column
print(matrix[0:2, 1:3])
```

Update a value:

```
matrix[0, 0] = 999
```

Update a complete column:

```
matrix[:, 1] = 0
```

7. Boolean and Fancy Indexing — Day 8

Boolean indexing

```
arr = np.array([10, 20, 30, 40, 50])

result = arr[arr > 25]

print(result)
```

Multiple conditions:

```
result = arr[
    (arr >= 20) &
    (arr <= 40)
]
```

Fancy indexing

```
positions = [0, 2, 4]

print(arr[positions])
```

where()

```
result = np.where(
    arr >= 30,
    "High",
    "Low"
)

print(result)
```

Replace negative values:

```
values = np.array([-10, 20, -5, 30])

clean_values = np.where(
    values < 0,
    0,
    values
)
```

8. Reshaping and Transpose — Day 9

Reshape:

```
arr = np.arange(1, 13)

matrix = arr.reshape(3, 4)

print(matrix)
```

Automatic dimension:

```
matrix = arr.reshape(2, -1)

print(matrix.shape)
```

Flatten:

```
flat_copy = matrix.flatten()
flat_view = matrix.ravel()
```

Transpose:

```
print(matrix.T)
```

Add a new dimension:

```
arr = np.array([1, 2, 3])

row = arr[np.newaxis, :]
column = arr[:, np.newaxis]

print(row.shape)
print(column.shape)
```

Remove dimensions of size one:

```
result = np.squeeze(row)
```

9. Combining and Splitting Arrays — Day 10

Concatenate:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

combined = np.concatenate([a, b])

print(combined)
```

Stack vertically:

```
print(np.vstack([a, b]))
```

Stack horizontally:

```
print(np.hstack([a, b]))
```

Create a new dimension:

```
print(np.stack([a, b], axis=0))
```

Split:

```
arr = np.arange(12)

print(np.split(arr, 3))
print(np.array_split(arr, 5))
```

Phase 3: Numerical Analysis

10. Mathematical Operations — Day 11

```
arr = np.array([10, 20, 30])

print(arr + 5)
print(arr - 5)
print(arr * 2)
print(arr / 2)
print(arr ** 2)
```

Array-to-array operations:

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Mathematical functions:

```
print(np.sqrt(a))
print(np.square(a))
print(np.abs(a))
print(np.log(a))
print(np.exp(a))
print(np.round(a / 3, 2))
```

11. Axis Concept — Day 12

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

Total sum:

```
print(np.sum(matrix))
```

Column-wise:

```
print(np.sum(matrix, axis=0))
```

Output:

```
[5 7 9]
```

Row-wise:

```
print(np.sum(matrix, axis=1))
```

Output:

```
[ 6 15]
```

Easy rule:

- **axis=0**: One result for each column

- **axis=1**: One result for each row

Keep dimensions:

```
result = np.sum(
    matrix,
    axis=1,
    keepdims=True
)

print(result.shape)
```

12. Aggregation and Statistics — Days 13–14

```
arr = np.array([10, 20, 30, 40, 50])

print(np.sum(arr))
print(np.mean(arr))
print(np.median(arr))
print(np.min(arr))
print(np.max(arr))
print(np.std(arr))
print(np.var(arr))
print(np.percentile(arr, 75))
print(np.quantile(arr, 0.75))
```

Find positions:

```
print(np.argmin(arr))
print(np.argmax(arr))
```

Cumulative operations:

```
print(np.cumsum(arr))
print(np.cumprod(arr))
print(np.diff(arr))
```

Correlation:


```
experience = np.array([1, 2, 3, 4, 5])
salary = np.array([30000, 40000, 50000, 60000, 70000])

print(np.corrcoef(experience, salary))
print(np.cov(experience, salary))
```

Weighted average:

```
marks = np.array([80, 70, 90])
weights = np.array([0.3, 0.2, 0.5])

result = np.average(
    marks,
    weights=weights
)

print(result)
```

13. Sorting and Searching — Day 15

Sort values:

```
arr = np.array([40, 10, 30, 20])

print(np.sort(arr))
print(np.sort(arr)[::-1])
```

Get sorting positions:

```
positions = np.argsort(arr)

print(positions)
print(arr[positions])
```

Search in sorted data:

```
sorted_arr = np.array([10, 20, 30, 40])

position = np.searchsorted(
    sorted_arr,
    25
)

print(position)
```

Unique values and counts:

```
arr = np.array([10, 20, 10, 30, 20, 10])

values, counts = np.unique(
    arr,
    return_counts=True
)

print(values)
print(counts)
```

Set operations:

```
a = np.array([1, 2, 3, 4])
b = np.array([3, 4, 5, 6])

print(np.union1d(a, b))
print(np.intersect1d(a, b))
print(np.setdiff1d(a, b))
print(np.isin(a, b))
```

14. Missing Values — Day 16

```
arr = np.array([
    10,
    20,
    np.nan,
    40
])
```

Find missing values:

```
print(np.isnan(arr))
```

Remove them:

```
clean = arr[~np.isnan(arr)]
```

Replace them:

```
filled = np.nan_to_num(  
    arr,  
    nan=0  
)
```

Calculate while ignoring missing values:

```
print(np.nanmean(arr))  
print(np.nanmedian(arr))  
print(np.nansum(arr))  
print(np.nanmin(arr))  
print(np.nanmax(arr))
```

Handle infinity:

```
values = np.array([  
    10,  
    np.inf,  
    -np.inf,  
    20  
)  
  
print(np.isinf(values))  
print(np.isfinite(values))
```

Phase 4: Advanced Array Calculations

15. Broadcasting — Days 17–18

Scalar broadcasting:

```
arr = np.array([10, 20, 30])  
  
print(arr + 5)
```

Row broadcasting:

```
matrix = np.array([  
    [10, 20, 30],  
    [40, 50, 60]  
)  
  
row = np.array([1, 2, 3])  
  
print(matrix + row)
```

Column broadcasting:

```
column = np.array([  
    [100],  
    [200]  
)  
  
print(matrix + column)
```

Broadcasting rules compare shapes from the right.

Dimensions are compatible when:

- They are equal.
- One dimension is 1.
- One array has no corresponding dimension.

Compatible:

```
(2, 3)  
(3,)
```

Compatible:

```
(2, 3)  
(2, 1)
```

Not compatible:

```
(2, 3)
(2,)
```

Practical feature scaling:

```
X = np.array([
    [10, 100],
    [20, 200],
    [30, 300]
])

mean = X.mean(axis=0)
std = X.std(axis=0)

X_scaled = (X - mean) / std
```

16. Random-Number Generation — Day 19

Use the modern generator:

```
rng = np.random.default_rng(42)
```

Random decimal values:

```
print(rng.random(5))
print(rng.random((2, 3)))
```

Random integers:

```
print(rng.integers(
    low=1,
    high=11,
    size=5
))
```

Normal distribution:

```
data = rng.normal(  
    loc=100,  
    scale=15,  
    size=1000  
)
```

Random selection:

```
colors = np.array([  
    "Red",  
    "Blue",  
    "Green"  
)  
  
sample = rng.choice(  
    colors,  
    size=5,  
    replace=True  
)
```

Shuffle:

```
arr = np.array([1, 2, 3, 4, 5])  
  
rng.shuffle(arr)
```

Shuffled copy:

```
shuffled = rng.permutation(arr)
```

17. Matrix Operations — Day 20

```
A = np.array([  
    [1, 2],  
    [3, 4]  
)  
  
B = np.array([  
    [5, 6],  
    [7, 8]  
)
```

Element-wise multiplication:

```
print(A * B)
```

Matrix multiplication:

```
print(A @ B)  
print(np.matmul(A, B))
```

Transpose:

```
print(A.T)
```

Determinant:

```
print(np.linalg.det(A))
```

Inverse:

```
print(np.linalg.inv(A))
```

Identity check:

```
inverse = np.linalg.inv(A)  
  
print(A @ inverse)
```

18. Linear Algebra — Day 21

Solve equations:

$$\begin{bmatrix} 3x+y=9 \end{bmatrix}$$

$$\begin{bmatrix} x+2y=8 \end{bmatrix}$$

```
A = np.array([
    [3, 1],
    [1, 2]
], dtype=float)

b = np.array([9, 8], dtype=float)

solution = np.linalg.solve(A, b)

print(solution)
```

Matrix rank:

```
print(np.linalg.matrix_rank(A))
```

Eigenvalues and eigenvectors:

```
symmetric_matrix = np.array([
    [2, 1],
    [1, 2]
], dtype=float)

values, vectors = np.linalg.eigh(
    symmetric_matrix
)

print(values)
print(vectors)
```

Singular Value Decomposition:

```
U, singular_values, Vt = np.linalg.svd(A)

print(U)
print(singular_values)
print(Vt)
```

Vector length:

```
vector = np.array([3, 4])

print(np.linalg.norm(vector))
```


Least-squares regression:

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])

X = np.column_stack([
    np.ones(len(x)),
    x
])

weights = np.linalg.lstsq(
    X,
    y,
    rcond=None
)[0]

print(weights)
```

19. Copy and View — Day 22

Assignment:

```
original = np.array([10, 20, 30])

another = original
another[0] = 999

print(original)
```

Both variables refer to the same array.

View:

```
original = np.array([10, 20, 30, 40])

view = original[1:3]
view[0] = 999

print(original)
```

Copy:

```
original = np.array([10, 20, 30, 40])

copied = original.copy()
copied[0] = 999

print(original)
print(copied)
```

Check shared memory:

```
print(np.shares_memory(original, copied))
```

Interview point:

- Basic slicing usually creates a view.
- Fancy indexing usually creates a copy.
- `copy()` creates independent data.

20. File Input and Output — Day 23

Save one array:

```
arr = np.array([10, 20, 30])

np.save("numbers.npy", arr)
```

Load:

```
loaded = np.load("numbers.npy")

print(loaded)
```

Save multiple arrays:

```
np.savez(
    "dataset.npz",
    features=X,
    target=y
)
```

Load multiple arrays:

```
data = np.load("dataset.npz")

print(data["features"])
print(data["target"])
```

CSV file:

```
np.savetxt(
    "numbers.csv",
    A,
    delimiter=",",
    fmt="%.2f"
)

loaded = np.loadtxt(
    "numbers.csv",
    delimiter=",",
)
```

CSV containing missing values:

```
loaded = np.genfromtxt(
    "data.csv",
    delimiter=",",
    skip_header=1,
    missing_values="",
    filling_values=np.nan
)
```

Phase 5: Performance and Machine Learning

21. Vectorization and Performance — Days 24–25

Slow loop:

```
arr = np.arange(1_000_000)

result =

for value in arr:
    result.append(value * 2)
```

Vectorized NumPy:

```
result = arr * 2
```

Avoid repeated `append()`:

```
# Slow
result = np.array()

for number in range(1000):
    result = np.append(result, number)
```

Better:

```
result = np.arange(1000)
```

Use suitable data types:

```
large = np.arange(
    1_000_000,
    dtype=np.int64
)

smaller = np.arange(
    1_000_000,
    dtype=np.int32
)

print(large.nbytes)
print(smaller.nbytes)
```

Use in-place operations carefully:

```
arr = np.array(
    [1.0, 2.0, 3.0]
)

arr *= 2
```

Time operations:

```
from timeit import timeit

time_taken = timeit(
    "arr * 2",
    setup="import numpy as np; arr=np.arange(1000000)",
    number=100
)

print(time_taken)
```

22. Advanced Ufuncs and `einsum()` — Day 26

Reduce:

```
arr = np.array([1, 2, 3, 4])

print(np.add.reduce(arr))
```

Accumulate:

```
print(np.add.accumulate(arr))
```

Outer operation:

```
a = np.array([1, 2, 3])
b = np.array([10, 20])

print(np.multiply.outer(a, b))
```

Count repeated positions:

```
indices = np.array([0, 1, 1, 2, 2, 2])
counts = np.zeros(3, dtype=int)

np.add.at(counts, indices, 1)

print(counts)
```

Matrix multiplication with `einsum()`:

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

result = np.einsum(
    "ij,jk->ik",
    A,
    B
)

print(result)
```

Dot product:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

dot_product = np.einsum(
    "i,i->",
    a,
    b
)

print(dot_product)
```

23. NumPy for Machine Learning — Days 27–28

Standardization

$[z = \frac{x - \mu}{\sigma}]$

```
X = np.array([
    [10, 100],
    [20, 200],
    [30, 300]
], dtype=float)

X_standardized = (
    X - X.mean(axis=0)
) / X.std(axis=0)

print(X_standardized)
```

Min-max normalization

[$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$]

```
X_normalized = (
    X - X.min(axis=0)
) / (
    X.max(axis=0) -
    X.min(axis=0)
)

print(X_normalized)
```

Mean squared error

```
actual = np.array([100, 200, 300])
predicted = np.array([110, 190, 310])

mse = np.mean(
    (actual - predicted) ** 2
)

print(mse)
```

Mean absolute error

```
mae = np.mean(
    np.abs(actual - predicted)
)
```

One-hot encoding

```
labels = np.array([0, 2, 1, 2])

one_hot = np.eye(3)[labels]

print(one_hot)
```

ReLU

```
values = np.array([-5, -1, 0, 3, 8])

relu = np.maximum(0, values)

print(relu)
```

Sigmoid

```
sigmoid = 1 / (
    1 + np.exp(-values)
)

print(sigmoid)
```

Softmax

```
scores = np.array([2.0, 1.0, 0.1])

stable_scores = scores - scores.max()

softmax = (
    np.exp(stable_scores) /
    np.exp(stable_scores).sum()
)

print(softmax)
```


Phase 6: Complete Numerical Projects

24. Complete Projects — Days 29–30

Recommended projects:

1. Student marks analysis
2. Employee salary analysis
3. Crop-yield numerical analysis
4. Image array manipulation
5. Linear-regression implementation
6. Sales trend analysis
7. Random dice simulation

8. Matrix calculator

Complete Crop-Yield Project

```
import numpy as np

# Columns:
# rainfall, temperature, fertilizer, crop_yield
data = np.array([
    [800, 25, 100, 3.2],
    [950, 24, 120, 3.8],
    [700, 29, 90, 2.7],
    [1100, 23, 140, 4.5],
    [650, 31, 80, 2.4],
    [1000, 26, 130, 4.0]
], dtype=float)

# Separate features and target
X = data[:, :3]
y = data[:, 3]

# Understand data
print("Data shape:", data.shape)
print("Feature shape:", X.shape)
print("Target shape:", y.shape)

# Statistical analysis
print("Feature means:", X.mean(axis=0))
print("Minimum values:", X.min(axis=0))
print("Maximum values:", X.max(axis=0))
print("Average yield:", y.mean())
print("Highest yield:", y.max())

# Best farm record
best_position = np.argmax(y)
best_record = data[best_position]

print("Best record:", best_record)

# High-yield farms
high_yield_farms = data[y >= 4.0]

print("High-yield farms:")
print(high_yield_farms)

# Standardize features
feature_mean = X.mean(axis=0)
feature_std = X.std(axis=0)

X_scaled = (
```

```
X - feature_mean
) / feature_std

# Create regression design matrix
design_matrix = np.column_stack([
    np.ones(X_scaled.shape[0]),
    X_scaled
])

# Train linear regression
weights = np.linalg.lstsq(
    design_matrix,
    y,
    rcond=None
)[0]

# Generate predictions
predictions = design_matrix @ weights

# Evaluate model
mse = np.mean(
    (y - predictions) ** 2
)

mae = np.mean(
    np.abs(y - predictions)
)

print("Model weights:", weights)
print("Predictions:", predictions)
print("MSE:", mse)
print("MAE:", mae)

# Predict a new farm
new_farm = np.array([
    900, 27, 110
], dtype=float)

new_farm_scaled = (
    new_farm - feature_mean
) / feature_std

new_design = np.column_stack([
    np.ones(new_farm_scaled.shape[0]),
    new_farm_scaled
])

predicted_yield = new_design @ weights

print("New farm predicted yield:", predicted_yield)
```

Final Target

After completing this roadmap, you should be able to:

- Create and manipulate multidimensional arrays.
- Select data using normal, Boolean and fancy indexing.
- Perform mathematical and statistical calculations.
- Understand **axis**, shape and broadcasting.
- Work with matrices and linear algebra.
- Generate reproducible random data.
- Handle missing values.
- Understand copies, views and memory.
- Replace slow loops with vectorized calculations.
- Implement preprocessing and ML formulas.
- Complete numerical data-analysis projects.